

Chapter 41: Authentication

1. Overview

Almost every real Angular application eventually needs to answer three questions: *who is this user*, *are they still allowed to be here*, and *how do we prove it on every request without asking them to log in again*. Authentication in an Angular SPA is not just "check a password" — it is a set of interlocking concerns:

- **Token-based auth** — issuing a short-lived access token (usually a JWT) and a longer-lived refresh token so the server stays stateless (or nearly so).
- **Storage strategy** — where the tokens live in the browser, and what that choice costs you in XSS/CSRF exposure.
- **Transport** — how every outgoing HTTP request gets the token attached, transparently, via an `HttpInterceptor`.
- **Silent refresh** — when the access token expires (typically a 401 response), the app must refresh it *once*, queue any other requests that failed for the same reason, and retry them — all without the user noticing or being logged out unnecessarily.
- **OAuth2 / OIDC** — when authentication is delegated to a third-party identity provider (IdP) such as Auth0, Okta, Azure AD, or a custom auth server, the SPA can't safely hold a `client_secret`, so it uses the **Authorization Code + PKCE** flow instead of the legacy Implicit flow.
- **Session lifecycle** — login, logout, expiry, and multi-tab consistency.
- **Route protection** — functional guards that gate navigation based on current auth state.
- **Auth state as a reactive primitive** — exposing "is logged in" / "current user" as a `Signal` or `observable` so every component and guard reads from one source of truth instead of polling `localStorage`.

This chapter builds a production-shaped authentication layer end-to-end: the token model, the storage tradeoffs, the interceptor pipeline (including the tricky refresh-queuing problem), PKCE mechanics, and the guard/signal integration that ties it all to the router.

2. Core Concepts

2.1 JWT Structure

A JSON Web Token is a compact, URL-safe string made of three Base64URL-encoded segments separated by dots:

```
header.payload.signature
```

- **Header** — algorithm and token type, e.g. `{"alg":"HS256","typ":"JWT"}`.
- **Payload (claims)** — the actual data. Standard/registered claims include:
 - `sub` — subject (user id)
 - `iat` — issued-at timestamp

- `exp` — expiry timestamp (Unix seconds)
- `nbf` — not-before
- `iss` — issuer
- `aud` — audience
- Plus custom claims (`roles`, `email`, `tenantId`, etc.)
- **Signature** — `HMACSHA256(base64url(header) + "." + base64url(payload), secret)` (or an asymmetric signature with RS256/ES256). The signature lets a server (or, for verification-only purposes, even the client) prove the payload wasn't tampered with, **but it does not encrypt the payload** — anyone can Base64-decode a JWT and read its claims. Never put secrets in a JWT payload.

Key interview point: **a JWT is signed, not encrypted**. Confidentiality of claims is not a property you get for free — if you need to hide claim contents from the client, you need a JWE (encrypted) or simply keep sensitive data server-side and return only an opaque reference.

Access tokens are typically short-lived (5–15 minutes) so that a leaked token has a small blast radius. Refresh tokens are long-lived (days to weeks) and are the credential actually protected with the most care, because a stolen refresh token lets an attacker mint new access tokens indefinitely.

2.2 Access Token vs Refresh Token

	Access Token	Refresh Token
Lifetime	Short (minutes)	Long (days/weeks)
Sent on	Every API request (<code>Authorization: Bearer <token></code>)	Only to the token endpoint, to mint a new access token
Contains	User identity + claims (JWT, often)	Often opaque, random, stored/looked-up server-side
Exposure if leaked	Limited (expires soon)	Severe (long window, may issue new tokens)
Revocable before expiry?	Hard (stateless JWTs can't be revoked without a blacklist)	Yes, server can delete/rotate it

2.3 Where to Store Tokens: `localStorage` vs `httpOnly` Cookies

This is one of the most-asked interview questions in this chapter, and the honest answer is "it depends on your threat model," not a one-line verdict.

Concern	<code>localStorage</code> / <code>sessionStorage</code>	<code>httpOnly</code> Secure Cookie
Accessible to JS	Yes — any injected script can read it	No — invisible to <code>document.cookie</code> and JS entirely

Concern	<code>localStorage</code> / <code>sessionStorage</code>	<code>httpOnly</code> Secure Cookie
XSS impact	High — a single XSS hole exfiltrates the token directly	Low — XSS can still perform actions as the user via the browser (cookie auto-attaches), but can't steal the token to replay elsewhere
CSRF impact	None (token isn't auto-attached; you attach it manually via interceptor)	Requires CSRF defense — cookie is sent automatically on cross-site requests unless <code>SameSite</code> is set
Cross-domain API (different origin than app)	Easy — you read the token and set the header yourself	Harder — needs <code>SameSite=None; Secure</code> and CORS with <code>credentials: 'include'</code> , and the API must be on a domain that can share cookies (or a <code>__Host-</code> cookie shared appropriately)
Works with mobile/native shells, non-browser clients	Yes	Cookies are browser-specific; awkward for native apps
Manual attach needed	Yes — interceptor reads storage and sets header	No — browser attaches automatically
Survives page refresh	Yes	Yes
Logout across tabs	Needs <code>storage</code> event listener	Cookie deletion is per-domain; still needs a broadcast mechanism for in-memory app state

Practical guidance commonly given in interviews:

- If you control both frontend and backend and they can share a cookie domain, prefer `httpOnly; Secure; SameSite=Strict|Lax` cookies for the **refresh token**, and keep the **access token in memory** (a plain in-memory variable, or a signal, not storage) — this way even XSS can't read the refresh token, and the access token disappears when the tab closes (and is short-lived anyway).
- If you must integrate with a third-party OAuth/OIDC provider issuing tokens directly to the SPA (no backend-for-frontend), you often don't have a choice about cookies, and you rely on: short access-token lifetime, `Content-Security-Policy` to reduce XSS surface, and, if the IdP supports it, storing tokens in memory only (not `localStorage`) to reduce persistence of a stolen token across reloads (at the cost of losing the session on refresh, mitigated by silent renewal iframes or refresh token rotation).
- `localStorage` is easy and common in tutorials, but purely from a security standpoint it is the weakest option against XSS. It remains extremely common in real production apps because most apps' real risk is dominated by other factors, and the ergonomics (simple SPA-to-API-on-another-domain, no CSRF plumbing) are attractive.

2.4 Attaching Tokens via Interceptors

Angular's `HttpInterceptorFn` (functional interceptors, Angular 15+) is the standard place to attach the `Authorization` header to outgoing requests, centralizing this logic instead of repeating it at every call site. Interceptors are also where you detect a 401 and kick off the refresh flow (Section 3).

2.5 Silent Refresh & Request Queuing — The Problem

When the access token expires mid-session:

1. Several HTTP calls might be in flight or fired in quick succession (e.g., a dashboard that fires 5 parallel requests).
2. All 5 come back `401 unauthorized` around the same time.
3. Naively, each 401 handler would call the refresh-token endpoint — resulting in **5 concurrent refresh calls**. Best case this is wasteful; worst case, if the server implements refresh-token rotation (each use invalidates the old token and issues a new one), only the *first* refresh call succeeds and the other 4 refresh calls fail because the refresh token they're holding was just invalidated by call #1 — effectively logging the user out despite having a technically valid session.

The fix is a **single-flight refresh gate**: the first 401 triggers the refresh; every other concurrent 401 is queued (not re-triggering refresh) and, once the refresh resolves, all queued requests are retried with the new token. This is covered in depth in Section 3 and Section 4.

2.6 OAuth2 / OIDC: Authorization Code + PKCE

For SPAs authenticating against a third-party or centralized identity provider, the modern, recommended flow is **Authorization Code with PKCE** (Proof Key for Code Exchange, RFC 7636). The old **Implicit flow** (tokens returned directly in the URL fragment) is deprecated for SPAs because it exposes tokens in browser history/referrers and has no client authentication step at all.

Why PKCE, and not a plain Authorization Code flow? A confidential backend client can hold a `client_secret` to prove, at the token exchange step, that *it* is the one that requested the code. A public client (a SPA, running entirely in the browser) cannot keep a secret — anything shipped to the browser is inspectable. PKCE replaces the static secret with a **dynamically generated, per-authorization-request proof**.

PKCE flow, step by step:

1. **Generate `code_verifier`** — the SPA generates a high-entropy random string (43–128 chars) client-side, kept only in memory/sessionStorage for the duration of the flow.
2. **Derive `code_challenge`** — `code_challenge = BASE64URL(SHA256(code_verifier))` (method `S256`; `plain` exists but should be avoided).
3. **Redirect to authorization endpoint** — the SPA redirects the browser to the IdP's `/authorize` endpoint with `response_type=code`, `client_id`, `redirect_uri`, `scope`, `state` (CSRF protection for the redirect), and `code_challenge + code_challenge_method=S256`. Note: the `code_verifier` is **not** sent at this step.
4. **User authenticates** at the IdP (login form, MFA, consent screen).
5. **IdP redirects back** to `redirect_uri` with `?code=...&state=...`. The SPA validates `state` matches what it sent (mitigates CSRF/mix-up attacks).
6. **Token exchange** — the SPA calls the IdP's `/token` endpoint with `grant_type=authorization_code`, the `code`, `redirect_uri`, `client_id`, and — critically — the original `code_verifier` (in plaintext, over HTTPS).
7. **IdP verifies**: it hashes the received `code_verifier` with SHA-256 and checks it equals the `code_challenge` submitted in step 3. Only if they match does it issue the access/refresh/ID tokens.

Why this stops interception: if an attacker intercepts the redirect in step 5 and steals the authorization `code` (e.g., via a malicious app registered on the same custom URL scheme on mobile, or a logged referrer), they still cannot exchange it for tokens, because they never had the `code_verifier` — it never left the legitimate SPA instance that generated it in step 1. The code alone is useless without the verifier.

The **ID Token** (OIDC, not OAuth2 proper) is a JWT that represents *authentication* (proof of identity, `nonce` - protected against replay) and is distinct from the **access token**, which represents *authorization* (what APIs the token can call). Don't send the ID token to your API as a bearer token — it's meant for the client to read the user's identity claims.

2.7 Logout / Session Expiry

- **Logout** should: clear local token storage/memory, notify the server to revoke the refresh token (so it can't be replayed even if leaked), redirect to login (or the IdP's end-session endpoint for full OIDC logout / SSO logout), and broadcast the logout to other open tabs.
- **Session expiry** (refresh token itself expired or revoked) should be detected when the refresh call itself fails with 401 — at that point, there's no recovery; force logout.
- **Clock skew** matters when checking `exp` client-side for a "proactively refresh before expiry" strategy — see Edge Cases.

2.8 Route Guards & Reactive Auth State

Modern Angular (14+) prefers **functional guards** (`CanActivateFn`) over class-based guards, and auth state is best modeled as a `Signal<boolean>` or `signal<User | null>` (or a `BehaviorSubject`/computed observable) so:

- Guards can synchronously/reactively check state via `inject(AuthService)`.
- Templates can `@if (auth.isLoggedIn())` without manual subscriptions.
- Any part of the app reacts immediately when auth state changes (e.g., hiding a "Login" button the instant a token expires).

3. Code Examples

3.1 Signal-Based `AuthService`

```
import { Injectable, signal, computed, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { Observable, tap, catchError, of, BehaviorSubject } from 'rxjs';

export interface AuthTokens {
  accessToken: string;
  refreshToken: string;
  expiresAt: number; // epoch ms
}

export interface User {
  id: string;
```

```

email: string;
roles: string[];
}

@Injectables({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private router = inject(Router);

  // --- Reactive auth state, exposed as read-only signals ---
  private _accessToken = signal<string | null>(null);
  private _user = signal<User | null>(null);

  readonly user = this._user.asReadonly();
  readonly isLoggedIn = computed(() => this._user() !== null);
  readonly accessToken = this._accessToken.asReadonly();

  // Used internally by the refresh interceptor as a single-flight gate.
  // null = no refresh in progress; otherwise holds the in-flight refresh Observable's
  result subject.
  private refreshInProgress$ = new BehaviorSubject<string | null>(null);

  constructor() {
    // Rehydrate from a persisted refresh token (httpOnly cookie handles this
    // automatically server-side; here we just attempt a silent refresh on boot).
    this.tryRestoreSession();
  }

  private tryRestoreSession(): void {
    this.http.post<{ accessToken: string; expiresAt: number; user: User }>(
      '/api/auth/refresh',
      {},
      { withCredentials: true } // sends the httpOnly refresh-token cookie
    ).pipe(
      tap(res => this.setSession(res.accessToken, res.expiresAt, res.user)),
      catchError(() => of(null))
    ).subscribe();
  }

  login(email: string, password: string): Observable<User> {
    return this.http.post<{ accessToken: string; expiresAt: number; user: User }>(
      '/api/auth/login',
      { email, password },
      { withCredentials: true }
    ).pipe(
      tap(res => this.setSession(res.accessToken, res.expiresAt, res.user)),
      // map to just the user for the caller
      tap(() => {}),
      catchError(err => { throw err; }),
      // return the user
      // (kept explicit for clarity rather than chaining map)
      tap(res => res.user),
      catchError(err => { throw err; }),
    );
  }
}

```

```

    // final transform
    // eslint-disable-next-line
    tap(),
    // simplest: use map in real code
    // (left verbose intentionally to show each step)
    tap(res => res.user)
  ) as unknown as Observable<User>;
}

logout(): void {
  this.http.post('/api/auth/logout', {}, { withCredentials: true }).subscribe({
    complete: () => this.clearSession(true),
    error: () => this.clearSession(true),
  });
}

/** Called by the refresh interceptor. Ensures only ONE network call happens
 * even if many 401s arrive concurrently. */
refreshAccessToken(): Observable<string> {
  if (this.refreshInProgress$.value === 'PENDING') {
    // Someone else already kicked off a refresh; wait for it to resolve.
    return this.refreshInProgress$.pipe(
      // filter out the PENDING/null placeholder, only emit the real token
      // (see interceptor for the full filter+take(1) usage)
    ) as unknown as Observable<string>;
  }

  this.refreshInProgress$.next('PENDING');

  return this.http.post<{ accessToken: string; expiresAt: number; user: User }>(
    '/api/auth/refresh',
    {},
    { withCredentials: true }
  ).pipe(
    tap(res => {
      this.setSession(res.accessToken, res.expiresAt, res.user);
      this.refreshInProgress$.next(res.accessToken);
    }),
    catchError(err => {
      this.refreshInProgress$.next(null);
      this.clearSession(true);
      throw err;
    }),
    tap(res => res.accessToken)
  ) as unknown as Observable<string>;
}

private setSession(accessToken: string, expiresAt: number, user: User): void {
  this._accessToken.set(accessToken);
  this._user.set(user);
  // Access token deliberately NOT persisted to localStorage – kept in memory.
  // Refresh token lives only in the httpOnly cookie set by the server.
}

```

```

private clearSession(broadcast: boolean): void {
  this._accessToken.set(null);
  this._user.set(null);
  if (broadcast) {
    // Notify other tabs (see Edge Cases: multi-tab logout).
    localStorage.setItem('logout-broadcast', Date.now().toString());
  }
  this.router.navigateByUrl('/login');
}

/** Client-side expiry check with a safety skew margin. */
isTokenExpiringSoon(expiresAt: number, skewMs = 10_000): boolean {
  return Date.now() >= expiresAt - skewMs;
}
}

```

Note: the `login/refreshAccessToken` bodies above are intentionally spelled out step-by-step for teaching clarity; in real code you'd simply use a single `map(res => res.user)` — see the cleaner version below used by the interceptor.

3.2 Auth Interceptor — Attaching the Bearer Token

```

import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from '../auth.service';

const API_PREFIX = '/api/';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.accessToken();

  // Don't attach the token to auth endpoints themselves (login/refresh) or
  // to third-party requests outside our API.
  const isAuthEndpoint = req.url.includes('/api/auth/');
  const isOurApi = req.url.startsWith(API_PREFIX) || req.url.includes('/api/');

  if (!token || isAuthEndpoint || !isOurApi) {
    return next(req);
  }

  const authReq = req.clone({
    headers: { Authorization: `Bearer ${token}` },
  });

  return next(authReq);
};

```

3.3 Refresh Interceptor — Queuing Concurrent 401s


```

import { HttpResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { BehaviorSubject, catchError, filter, switchMap, take, throwError } from 'rxjs';
import { AuthService } from '../auth.service';

// Module-level (singleton) gate shared across all requests going through
// this interceptor instance. `null` = idle, string = last-refreshed token
// currently valid to retry with, and we use a separate boolean flag to
// distinguish "idle" from "refresh in flight".
let isRefreshing = false;
const refreshedToken$ = new BehaviorSubject<string | null>(null);

export const refreshInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);

  return next(req).pipe(
    catchError((error: unknown) => {
      if (
        error instanceof HttpResponse &&
        error.status === 401 &&
        !req.url.includes('/api/auth/refresh') // never retry a failed refresh itself
      ) {
        if (!isRefreshing) {
          isRefreshing = true;
          refreshedToken$.next(null); // reset gate for new round

          return auth.refreshAccessToken().pipe(
            switchMap(newToken => {
              isRefreshing = false;
              refreshedToken$.next(newToken);
              // Retry the original request with the new token.
              const retried = req.clone({
                headers: { Authorization: `Bearer ${newToken}` },
              });
              return next(retried);
            }),
            catchError(refreshErr => {
              isRefreshing = false;
              refreshedToken$.next(null);
              // Refresh itself failed -> session is truly over.
              return throwError(() => refreshErr);
            })
          );
        }
      }
    })
  );
}

// A refresh is already in flight: wait for it instead of firing
// a second refresh call, then retry with whatever token results.
return refreshedToken$.pipe(
  filter((token): token is string => token !== null),
  take(1),
  switchMap(token => {
    const retried = req.clone({
      headers: { Authorization: `Bearer ${token}` },
    });

```

```

    });
    return next(retried);
  })
);
}

return throwError(() => error);
})
);
};

```

Register both interceptors, refresh interceptor typically *after* the auth interceptor in the chain (order in `provideHttpClient(withInterceptors([...]))` matters — request-side runs left-to-right, response-side runs right-to-left, but since both only act on request headers or response errors respectively, in practice put `authInterceptor` first so it stamps the token, and let `refreshInterceptor` react to 401s bubbling back through the chain):

```

import { ApplicationConfig } from '@angular/core';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from './auth.interceptor';
import { refreshInterceptor } from './refresh.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withInterceptors([authInterceptor, refreshInterceptor])),
  ],
};

```

3.4 Functional Auth Guard

```

import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from './auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;
  }

  // Preserve intended destination so we can redirect back after login.
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url },
  });
};

// Role-based variant, parametrized via a guard factory:
export const roleGuard = (allowedRoles: string[]): CanActivateFn => {
  return () => {

```

```

const auth = inject(AuthService);
const router = inject(Router);
const user = auth.user();

if (user && allowedRoles.some(r => user.roles.includes(r))) {
  return true;
}
return router.createUrlTree(['/forbidden']);
};
};

```

```

// routes.ts
import { Routes } from '@angular/router';
import { authGuard, roleGuard } from './auth.guard';

export const routes: Routes = [
  { path: 'login', loadComponent: () => import('./login.component') },
  {
    path: 'dashboard',
    canActivate: [authGuard],
    loadComponent: () => import('./dashboard.component'),
  },
  {
    path: 'admin',
    canActivate: [authGuard, roleGuard(['admin'])],
    loadComponent: () => import('./admin.component'),
  },
];

```

4. Internal Working

4.1 How the Single-Flight Refresh Gate Actually Prevents Parallel Refresh Calls

The core trick is a **module-scoped mutable flag** (`isRefreshing`) paired with a `BehaviorSubject` acting as a **broadcast/rendezvous point**:

1. Request A, B, C all fire near-simultaneously and all receive `401`.
2. Each one flows independently into the `catchError` operator of `refreshInterceptor` (interceptors run per-request, so there are three separate pipeline executions, not one).
3. **Request A** arrives first (order isn't guaranteed but exactly one wins the race for the synchronous check): `isRefreshing` is `false`, so it flips it to `true` **synchronously**, before any async work happens, and starts the actual `POST /refresh` call.
4. **Request B and C** arrive at the same `catchError` block microseconds later. Because `isRefreshing` is now `true` (set synchronously by A before A's HTTP call even resolves), B and C take the `else` branch: they do **not** call `refreshAccessToken()` again. Instead they subscribe to `refreshedToken$` and wait.
5. `refreshedToken$` is a `BehaviorSubject<string | null>`, initialized/reset to `null` right when the

refresh round starts. B and C use `filter(token => token !== null) + take(1)` — so they simply sit subscribed, doing nothing, until a non-null value arrives.

- When A's refresh call resolves, it calls `refreshedToken$.next(newToken)`. This single emission fans out to **every** subscriber waiting on that subject — B and C's `filter+take(1)` both fire, each retries its own original request with `newToken`, then auto-unsubscribes (`take(1)`).
- `isRefreshing` is reset to `false` so the *next* time a 401 occurs (a genuinely new expiry event later), a fresh refresh round can start.

The critical property making this safe is that **step 3's flag flip is synchronous** — there's no `await`/microtask gap between "check the flag" and "set the flag" for a single-threaded JS event loop, so there's no window where two requests both see `isRefreshing === false` and both start a refresh. (This would not be safe if `isRefreshing` were, say, read and written across separate async boundaries or in a multi-threaded context — but JS's single-threaded, run-to-completion execution of synchronous code guarantees no interleaving here.)

Why a `BehaviorSubject` instead of a plain `Subject`: it doesn't strictly need to be a `BehaviorSubject` (a plain `Subject` would work too since we use `filter+take(1)` and never need "replay to late subscribers" semantics mid-round) — but `BehaviorSubject` is convenient because it lets you synchronously reset state (`.next(null)`) at the start of a round without needing a separate "current value" holder, and it plays well if you ever want a late-arriving request (D, arriving just after the round completed but is a stale retry) to synchronously read the last-known token via `.value`.

If you skip this pattern and just call `refreshAccessToken()` from every failed request independently, and your backend implements **refresh token rotation** (each refresh both issues a new access token *and* invalidates+replaces the refresh token), you get a real bug: request A's refresh succeeds and rotates the refresh token; request B's refresh call, which raced in with the *old* (now-invalidated) refresh token, gets rejected by the server, and B's interceptor logic (naively) treats that as "session over" and force-logs-out the user — even though the user's session is perfectly valid, just because two refresh calls raced.

4.2 How PKCE's `code_verifier`/`code_challenge` Defeats Authorization Code Interception

The threat PKCE defends against is specifically **authorization code interception** — a scenario where an attacker, through some channel other than compromising the SPA itself (a malicious app intercepting a custom URI scheme redirect on mobile, a network intermediary, a referrer leak, or a compromised browser extension watching redirects), obtains the `code` value from step 5 of the flow.

Without PKCE (or a `client_secret`), possessing the `code` alone would be sufficient to call `/token` and receive access/refresh tokens — the attacker impersonates the legitimate app.

With PKCE:

- The `code_verifier` is generated **inside** the legitimate app instance and is never transmitted anywhere until the final token-exchange POST request (step 6), which goes over TLS directly from app to IdP — it never appears in a URL, browser history, referrer header, or redirect.
- The authorization endpoint (step 3) only ever receives the **derived** `code_challenge` (a one-way SHA-256 hash), not the verifier itself. Even if this request is observed, the challenge cannot be reversed back into the verifier (that's the point of using a cryptographic hash).
- The IdP binds the issued `code` to that specific `code_challenge` server-side.

- When *anyone* (legitimate app or attacker) presents that `code` to `/token`, the IdP demands the matching `code_verifier` and independently re-hashes it to check equality with the stored `code_challenge`.
- An attacker who only intercepted the `code` (from step 5's redirect) has no way to produce the correct `code_verifier` — they never had it and cannot derive it from the challenge or the code (hashing is one-way, and the verifier is high-entropy random, not guessable).

So PKCE converts "possession of the code" into "possession of the code **and** a secret generated at flow-start" as the requirement to redeem tokens, closing the gap that made public clients unsafe to use with plain Authorization Code flow.

5. Edge Cases & Gotchas

- **XSS and `localStorage`**: if *any* script on the page can execute arbitrary JS (a vulnerable third-party widget, an unsanitized `[innerHTML]` bound to user content, a compromised npm dependency), it can do `localStorage.getItem('accessToken')` and exfiltrate it to an attacker's server — full account takeover, no special privilege needed. This is why in-memory storage (or `httpOnly` cookies for the refresh token) is the security-preferred option, at the cost of losing the token on hard refresh unless you silently re-authenticate.
- **Race conditions during concurrent refresh** (see Section 4.1): the classic bug is firing `N` refresh calls for `N` concurrent 401s. Beyond wasted network calls, this actively breaks refresh-token-rotation backends. Always gate refresh behind a single in-flight guard.
- **Refresh token rotation and reuse detection**: many modern auth servers not only rotate the refresh token on every use but also treat *reuse of an already-consumed refresh token* as a signal of theft (someone replayed an old token) and respond by revoking the *entire token family* (all descendant tokens), forcing a full re-login. This makes the single-flight interceptor pattern not just an optimization but a correctness requirement — accidentally reusing a stale refresh token due to a race can nuke a legitimate session.
- **Clock skew on expiry checks**: if you proactively check `Date.now() >= tokenExpiresAt` client-side to decide whether to refresh *before* making a request (rather than reactively on 401), the client's clock and the server's clock may differ by seconds (or more, on misconfigured devices). Always apply a **skew margin** (e.g., treat the token as expired 10–30 seconds before its actual `exp`) so you refresh slightly early rather than sending a request with a token the server considers already expired.
- **Logging out across multiple tabs**: if tab A logs out (clears in-memory token/state) but tab B still has the old in-memory state, tab B will keep functioning as if logged in until its next failed request. The standard fix is a **storage event** broadcast: on logout, write a sentinel key to `localStorage` (as shown in `clearSession`); other tabs listen via `window.addEventListener('storage', ...)` (this event only fires in *other* tabs, not the one that made the write) and react by clearing their own state and redirecting to `/login`. `BroadcastChannel` is a cleaner, purpose-built modern alternative to the `storage`-event hack.
- **Silent refresh via hidden iframe (classic OIDC pattern)**: some IdPs support `prompt=none` redirects in a hidden iframe to silently renew tokens using the IdP's own session cookie, without a visible redirect. This is increasingly blocked by browsers' third-party cookie restrictions (Safari ITP, Chrome's phase-out), which is part of why refresh-token-based silent refresh (via your own backend) or a Backend-for-Frontend (BFF) pattern is now often preferred over iframe-based silent renew for SPAs.
- **state parameter reuse/omission in OAuth**: skipping or weakly generating the `state` parameter in

the Authorization Code flow reopens CSRF-style attacks on the redirect step (an attacker tricks a victim into completing an OAuth flow initiated by the attacker, binding the victim's session to the attacker's resource). PKCE protects the *code exchange*, `state` protects the *redirect callback* — they solve different problems and both are needed.

- **Token stored in a signal vs a plain field:** storing the access token in a `signal()` (as shown) rather than a plain class field lets templates and `computed()` /effects react automatically to login/logout/refresh, but be careful not to accidentally serialize/log the whole `AuthService` (e.g., via a naive Redux devtools/state debug integration) since that could leak the token into logs.
- **Guard timing vs. async auth restoration:** if `isLoggedIn` is only known after an async "restore session" HTTP call resolves (as in `tryRestoreSession()`), a synchronous guard checked at app boot might redirect to `/login` even though the user actually has a valid refresh-token cookie, simply because the restore call hasn't resolved yet. Guards should either await an app-initializer that blocks first navigation until the restore completes, or be written to return an `Observable<boolean>` / `Promise<boolean>` that waits on the auth-state resolution rather than reading a signal synchronously before it's populated.

6. Interview Questions & Answers

Q1. What's the difference between authentication and authorization?

Authentication answers "who are you" (verifying identity — login). Authorization answers "what are you allowed to do" (permissions/roles once identity is established). A JWT access token often carries both: identity claims (`sub`, `email`) and authorization claims (`roles`, `scope`).

Q2. Is a JWT encrypted?

No — a standard JWT (JWS) is signed, not encrypted. The header and payload are just Base64URL-encoded JSON, trivially decodable by anyone. The signature only guarantees integrity/authenticity (it wasn't tampered with and came from a holder of the signing key), not confidentiality. Never place secrets in JWT claims.

Q3. Why shouldn't you store JWTs in `localStorage`?

Interviewer intent: checks whether the candidate understands XSS as the dominant browser-storage threat, not just "cookies bad, storage good" folklore.

Because any script that can execute on the page — from a genuine XSS vulnerability, a compromised third-party dependency, or an injected browser extension — can read `localStorage` synchronously and exfiltrate the token, giving full account takeover with no extra steps. `httpOnly` cookies are invisible to JS, so the same XSS can still act on the user's behalf via automatic cookie attachment, but can't steal the token itself for offline replay elsewhere. The tradeoff is that cookies bring CSRF concerns and cross-origin complexity, which is why the answer is "it depends on your architecture," not an absolute rule.

Q4. How do you attach an auth token to every outgoing HTTP request in Angular?

Via an `HttpInterceptorFn` registered with `provideHttpClient(withInterceptors([authInterceptor]))`. The interceptor reads the current token (ideally from a signal/service, not directly from storage each time) and clones the outgoing request with `req.clone({ setHeaders: { Authorization: 'Bearer ' + token } })`, since `HttpRequest` objects are immutable.

Q5. Why must you clone the request in an interceptor instead of mutating it?

`HttpRequest` (and `HttpHeaders`) are immutable by design so that a request object can be safely shared, retried, and passed through multiple interceptors without one interceptor's mutation affecting another's view of it or breaking retries. `.clone()` returns a new request with the specified overrides.

Q6. What problem does silent-refresh request queuing solve, and how do you implement it?

Interviewer intent: this is the flagship "advanced" question of the chapter — tests whether the candidate has actually built this, not just read about JWTs.

When multiple requests fail with 401 around the same time (the token expired), naively refreshing once per failed request causes redundant network calls and, worse, can break backends using refresh-token rotation (a second refresh call using an already-consumed/rotated token gets rejected, incorrectly triggering logout). The fix is a single-flight gate: a shared flag/subject tracks whether a refresh is already in progress. The first 401 flips the flag, starts the refresh, and on success broadcasts the new token through a `BehaviorSubject / Subject`. Every other concurrent 401 checks the flag, sees a refresh already in flight, and instead of calling refresh again, subscribes to the shared subject (`filter(non-null) + take(1)`), retrying its own original request once the new token arrives.

Q7. Why is the flag-check-and-set in the refresh interceptor safe without an explicit lock?

Because JavaScript is single-threaded with run-to-completion semantics for synchronous code — checking `isRefreshing` and setting it to `true` happens in one synchronous block with no `await` /microtask boundary in between, so no other invocation of the interceptor can interleave and see a stale `false` value. This wouldn't hold if the check-then-set spanned an `await`.

Q8. What is PKCE and why does a SPA need it instead of a plain Authorization Code flow?

PKCE (Proof Key for Code Exchange) adds a dynamically generated `code_verifier` / `code_challenge` pair to the OAuth2 Authorization Code flow so that public clients (like SPAs, which cannot hold a `client_secret` safely since anything shipped to the browser is inspectable) can still prove, at token-exchange time, that they are the same entity that initiated the authorization request. Without it, anyone who intercepts the authorization `code` (from a redirect, referrer leak, or malicious app on a shared URI scheme) could redeem it for tokens themselves.

Q9. Walk through the PKCE flow.

1. SPA generates a random `code_verifier`.
2. SPA computes `code_challenge = BASE64URL(SHA256(code_verifier))`.
3. SPA redirects to the IdP's `/authorize` with `code_challenge`, `code_challenge_method=s256`, plus `client_id`, `redirect_uri`, `state`.
4. User authenticates at the IdP.
5. IdP redirects back with an authorization `code` (bound server-side to that `code_challenge`).
6. SPA POSTs to `/token` with the `code` plus the original `code_verifier`.
7. IdP hashes the received verifier and compares to the stored challenge; only on match does it issue tokens.

Q10. What's the difference between an ID token and an access token in OIDC?

The ID token is a JWT that asserts the user's *authentication* — who they are, when/how they logged in, protected against replay via a `nonce` — and is meant to be consumed by the client application itself, not sent to APIs. The access token grants *authorization* to call specific APIs/scopes and is what gets sent as a `Bearer` token to resource servers. Sending the ID token to your API as if it were an access token is a common and incorrect shortcut.

Q11. How do you protect a route based on auth state in Angular, using modern APIs?

With a functional `CanActivateFn` guard (`authGuard`) that injects the auth service and checks a signal/observable of login state, returning `true` to allow navigation or a `UrlTree` (via `router.createUrlTree(...)`) to redirect — returning a `UrlTree` instead of imperatively calling `router.navigate()` is preferred because the router handles it as a single coordinated navigation instead of two competing ones.

Q12. Why model auth state as a signal (or observable) rather than a plain boolean flag read from a service method?

Interviewer intent: probes understanding of reactivity vs. imperative polling, and whether change detection concerns are understood.

A plain method (`isLoggedIn(): boolean`) has to be manually re-invoked by change detection or explicit calls — components/templates won't automatically re-render when the underlying value changes unless you wire up subscriptions yourself. A `signal` / `computed` (or an `observable` consumed via the `async` pipe) makes auth state a reactive source: any template, guard, or computed value that reads it is automatically kept in sync the instant login/logout/refresh happens, with no manual change-detection triggering, and it composes cleanly with other signals (e.g., `computed(() => isLoggedIn() && hasRole('admin'))`).

Q13. How do you handle logout so it's consistent across multiple open tabs?

Clearing in-memory/service state in the tab that initiated logout isn't enough — other tabs still hold stale "logged in" state. The standard fix is to write a sentinel value to `localStorage` on logout and listen for the `storage` event (which only fires in *other* tabs) in every tab, so each tab reacts by clearing its own auth state and redirecting to the login page. `BroadcastChannel` is a more purpose-built modern alternative to the `storage`-event trick.

Q14. What is refresh token rotation, and what new failure mode does it introduce?

Interviewer intent: checks whether the candidate connects the security feature (rotation) to the client-side engineering consequence (single-flight requirement), which is exactly the kind of cross-cutting reasoning senior candidates are expected to show.

Rotation means every time a refresh token is used, the server invalidates it and issues a brand-new refresh token alongside the new access token — so a refresh token is single-use. This limits the damage of a leaked refresh token (it becomes useless after one legitimate use) and lets the server detect theft: if an already-used (rotated-away) refresh token is presented again, that's a strong signal of compromise, and many servers respond by revoking the entire token family, forcing re-login. The client-side consequence is that any race where two requests both attempt to use the "current" refresh token concurrently will have one succeed and one fail — which is exactly why the interceptor must serialize refresh calls through a single-flight gate rather than letting each failed request trigger its own independent refresh.

Q15. If your access token is stored only in memory (not persisted at all), what breaks on a hard page refresh, and how do you recover?

The in-memory signal/variable is wiped by the browser reload, so the SPA momentarily looks logged out. Recovery relies on the durable credential surviving the reload: a `httpOnly` refresh-token cookie (sent automatically) that the app calls on bootstrap (an `APP_INITIALIZER` /route resolver hitting `/api/auth/refresh`) to silently mint a new access token before rendering protected routes — or, in third-party OIDC setups without a backend, a silent-renew redirect/iframe against the IdP's session. The guard/router should wait for this bootstrap check to resolve rather than assuming "no in-memory token" means "not logged in."

7. Quick Revision Cheat Sheet

- **JWT** = header.payload.signature, Base64URL, **signed not encrypted** — never put secrets in claims.
- **Access token**: short-lived, sent as `Authorization: Bearer <token>` on every API call.
- **Refresh token**: long-lived, sent only to the token/refresh endpoint, the higher-value target for attackers.
- **Storage**: `localStorage` → simple, but full XSS-exfiltration risk. `httpOnly` cookie → immune to JS/XSS theft, but needs CSRF defense (`SameSite`) and CORS `credentials` . Common best practice: refresh token in `httpOnly` cookie, access token in memory only.
- **Interceptor pattern**: one interceptor attaches the bearer token; a second catches `401` , and refreshes.
- **Refresh queuing**: gate refresh behind a single in-flight flag + `BehaviorSubject / Subject` ; concurrent `401`s wait and retry, they never trigger a second refresh call. Essential when the backend does refresh-token rotation.
- **PKCE** = `code_verifier` (random secret, kept client-side) + `code_challenge = SHA256(code_verifier)` (sent to `/authorize`). Verifier only ever leaves the client at the final `/token` exchange, so an intercepted `code` alone is useless.
- **state parameter** protects the redirect step from CSRF; PKCE protects the code-exchange step from interception — different problems, both needed.
- **ID token** (OIDC) = who the user is, for the client. **Access token** (OAuth2) = what APIs can be called, sent to the resource server. Don't mix them up.
- **Guards**: use functional `CanActivateFn` , return `true` or a `UrlTree` redirect; base the check on reactive auth state (signal/observable), and make sure the guard waits for any async session-restore before deciding.
- **Auth state as signal/observable**: enables automatic reactivity everywhere (`computed` , templates via `@if / async` pipe) instead of manual polling or missed change-detection updates.
- **Logout**: clear client state + revoke refresh token server-side + broadcast to other tabs (`storage` event or `BroadcastChannel`) + redirect.
- **Clock skew**: always subtract a skew margin (10–30s) when proactively comparing `Date.now()` to a token's `exp` .
- **Multi-tab consistency** and **rotation-reuse detection** are the two gotchas interviewers most often probe beyond the "happy path" flow.

Created By - Durgesh Singh